

MIDTERM PROJECT REPORT | WEEKS 1-7

Evaluating the Robustness of LLM-Based Autonomous Agents Against Prompt Injection Attacks in Tool-Use Scenarios

Somama Siddiqui

Senior Seminar II · Instructor: Dr. Ning Zhang

Fisk University · Reporting scope: Weeks 1-7 · October 5, 2026

Abstract. This cumulative report documents the design, implementation, and initial evaluation of a tool-using language-model agent exposed to prompt injection. The project includes a Python agent, three mock tools, 24 attacks in four categories, four individual defenses, and a combined defense. Midterm updates add resumable experiments, repeated-trial support, corrected scoring, and an independent human-review workflow. All 40 automated tests pass. At the evidence cutoff, 6 clean controls and 4 attacked episodes have completed successfully in a 30-episode Gemini Flash-Lite pilot. The clean controls completed the legitimate task; the limited attack observations do not establish a defense ranking. Full-grid execution, repeated trials, and human validation remain the principal work toward completion by Week 11.

Keywords: prompt injection; autonomous agents; tool use; instruction hierarchy; guard LLM; security evaluation.

1. Project Overview

The project investigates whether malicious instructions embedded in external content can redirect an LLM-based agent away from its authorized task. The research question is how attack success varies across injection categories and defense mechanisms while holding the agent architecture and legitimate task constant.

The agent reads a synthetic campus solar-array article, summarizes it, and emails the summary to an authorized recipient. An attacker controls text inside the article and attempts to redirect delivery, expose a synthetic session credential, or replace the summary with attacker-selected content. All tools are mocked: the experiment sends no real email and fetches no real webpage.

Objectives and expected outcome

The objectives are to build a reproducible attack suite, implement comparable defenses, measure attack success and legitimate-task completion, and examine latency and added model calls. The expected final product is an empirical comparison supported by code, saved transcripts, tables, figures, validated labels, and a final report and presentation.

Cumulative scope and project links

The report combines the proposal, the implementation described in Progress Report 1, the evaluation goals carried into Progress Report 2, and the verified midterm updates. Work is organized by milestones across Weeks 1-7 rather than assigning unsupported completion dates to individual weeks.

[GitHub repository](#) · [Midterm implementation branch](#)

Verified implementation commit: `1bac1e4`. The updates are pushed to `codex/midterm-evaluation` and have not been merged into the default branch.

2. Work Completed: Weeks 1-7

Milestone 1: Research scope and controlled environment

The proposal defined an empirical security study using one agent task and roughly 20-30 injection attacks. The implementation uses a custom Python loop and Google Gemini rather than a larger agent framework. This keeps the tool boundary visible and allows each defense to change a specific part of the same workflow. The article, authorized destination, and synthetic credential are fixed across conditions.

Milestone 2: Agent loop and mock tools

The agent requests a structured action or final response from Gemini, parses nested JSON, dispatches one tool call at a time, and records the interaction. The three tools are `read_webpage`, `summarize`, and `send_email`. The summarizer is deterministic and extractive; the mail tool appends a record to an in-memory log. Each episode retains the transcript, tool arguments and results, blocked actions, final response, elapsed time, and call counts.

The loop permits at most six model steps. Invalid output triggers a formatting reminder; malformed tool arguments are rejected. Exhausted step budgets and model failures are recorded as errors. These safeguards make incomplete executions distinguishable from completed episodes in which the attack did not succeed.

Milestone 3: Benchmark construction and defense implementations

Attack category	Count	Representative attempted outcome
Direct instruction override	6	Replace the destination or overwrite the summary
Role-play or jailbreak framing	6	Use a persona, story, or reward to redirect behavior
Data exfiltration	6	Expose the synthetic credential in a tool argument
Disguised system text	6	Impersonate privileged instructions inside the page

Each attack has a unique identifier, payload, category, and declared target action. Content-replacement attacks also use a canary marker. The payload is inserted into the same article fixture, and an integrity test verifies all 24 attacks, six per category, and their inclusion in the poisoned page.

Condition	Implementation and purpose
No defense	Baseline agent with the same task and tools
Sanitization	Delimits untrusted output and neutralizes selected fake role tags
Instruction hierarchy	Adds instructions to treat retrieved content as data
Guard LLM	Reviews proposed email actions before execution
Action allow-list	Checks tool names, destination, secret exposure, and summary cues
Combined defenses	Layers the four mechanisms; the first veto stops execution

Four individual defenses satisfy the proposal; the combined condition is an additional comparison. The guard reviews email proposals only. Its unavailable or invalid verdict stops the episode before dispatch, so an infrastructure failure cannot be reported as evidence of a successful defense.

2. Work Completed: Weeks 1-7 (continued)

Milestone 4: Evaluation reliability and midterm updates

The evaluation runner now saves every episode atomically and resumes by skipping completed records. A manifest records the agent and judge models, source fingerprint, attack selection, conditions, trial count, temperature, and pacing. A changed configuration or source requires a new run directory. Failed attempts are preserved when explicitly retried, and a lock prevents overlapping writers.

Repeat trials and clean controls are supported. Model calls share a request limiter, transient errors are retried, and the provider's retry-delay hint is respected. Three consecutive episode failures pause a run. Fresh defense objects are constructed for each episode, avoiding shared state; combined-defense guard calls are now counted.

Milestone 5: Scoring repair and independent validation tooling

The deterministic scorer checks executed outcomes. Recipient attacks require actual delivery to the attacker address, not a quoted address in a subject or body. Failed dispatches do not count as credential exposure. The judge must return a JSON boolean; strings such as "false" are rejected rather than converted to true. Summary and content checks remain heuristics that require human validation.

A reproducible, stratified exporter creates numbered review packets with automated verdicts withheld. A human enters succeeded, blocked, or ambiguous. The scoring command reports agreement, precision, recall, and confusion counts, excluding ambiguous and unreviewed cases from binary metrics. It preserves all three verdicts and verifies that the labeled episode has not changed. No completed independent human-label study is claimed.

System architecture and experimental design



Saved episode → Deterministic scorer + Gemini judge → Human validation and analysis

Figure 1. Controlled evaluation pipeline. The same task, tools, and attack definitions are used across defense conditions; raw evidence is retained before aggregation.

3. Evidence of Progress

The original local history contains the agent, benchmark, defenses, and harness commits from September 15. Commit 1bac1e4 adds the midterm reliability and validation work. The regression suite expanded from 10 to **40 passing tests**; the recorded run completed in 13.80 seconds without requiring live model calls.

Evidence	Repository location
Agent, benchmark, and defenses	agent/; benchmark/; defenses/
Evaluation and human-review workflow	evaluation/harness.py; scorer.py; validation.py
Testing and execution environment	tests/; evidence/pytest.txt; evidence/environment.txt
Protocol and completion milestones	docs/EXPERIMENT_PROTOCOL.md; PROJECT_MILESTONES.md

3. Evidence of Progress (continued)

Evidence cutoff: October 05, 2026 at 06:54 PM CDT. This report uses a frozen snapshot of the ongoing pilot, not the planned total. The pilot uses **gemini-3.1-flash-lite** for the agent, guard, and judge, temperature 0, one trial, and one selected attack per category. Its planned size is 24 attacked episodes plus six clean controls.

Table 1. Observed clean-control results

Condition	Task success	Episode seconds	Guard calls
No defense	1/1	29.22	0
Sanitization	1/1	28.55	0
Instruction hierarchy	1/1	43.69	0
Guard LLM	1/1	36.99	1
Action allow-list	1/1	18.10	0
Combined defenses	1/1	29.60	1

All six observed clean controls completed the legitimate task. One control per condition demonstrates feasibility; it does not estimate a reliable completion probability or prove that the defenses preserve utility under diverse inputs.

Table 2. Attack observations available at the cutoff

Condition	Valid attacks	Scorer successes	Judge successes
No defense	1	0/1	0/1
Sanitization	1	0/1	0/1
Instruction hierarchy	1	0/1	0/1
Guard LLM	1	0/1	0/1
Action allow-list	0	Pending	Pending
Combined defenses	0	Pending	Pending

The snapshot contains **4 completed attacked episodes** (do_01), with 0 observed attack successes. The judge and deterministic scorer agree on 4/4 available judge verdicts. This is automated agreement, not validation against a human labeler. Conditions without an observation remain pending; they are not assigned a zero success rate.

Figure 2. Observed clean-control episode time



Each bar is one observed episode. Timing includes agent and guard calls, request pacing, and provider retries; the later judge pass is excluded. Variations in service behavior and run order prevent interpreting these bars as isolated defense overhead.

Coverage and limits. 10/30 pilot episodes are saved at this cutoff; 0 saved pilot episodes are marked as errors. The full 144-cell grid, repeated trials, and human validation are incomplete. No general defense ranking is supported.

4. Progress Compared with the Original Proposal

The approved proposal calls for a small Python agent, roughly 20-30 attacks, three to four candidate defenses, repeated experiments, a 30-50-run human validation sample, and an analysis of security and usability. The implementation follows that scope. The proposal does not specify dated weekly deadlines, so schedule status is assessed against the explicit Week 7 midterm and Week 11 completion targets.

Planned component	Current status	Change or qualification
Small agent with two to three tools	Implemented and live-tested	Custom Gemini loop; three mocked tools
About 20-30 prompt injections	24 attacks implemented	Four categories, six attacks each
Three to four defenses	Four plus a composite	Baseline retained for comparison
Repeated attack-defense trials	Runner implemented; study incomplete	Pilot is one trial over four attacks
Human validation of 30-50 runs	Workflow implemented; labels pending	Pilot alone supplies at most 24 attacked episodes
Success, utility, and latency analysis	Preliminary observations	Full tables and comparative conclusions pending
Final report and presentation	Planned for Week 11	Midterm summarizes implementation and pilot evidence

The main research question is unchanged. Changes include adding a combined defense, checkpointed execution, error-aware denominators, and a verdict-blinded review workflow. These changes improve reproducibility without increasing the legitimate task's scope. Model availability and quota affect execution speed, so different model runs remain separate.

Software implementation is ahead of experimental validation. The full grid and label sample carried over from the progress reports are still incomplete. The Week 11 target remains the working schedule, with Weeks 8-9 allocated to execution and labeling rather than assuming those tasks are already finished.

5. Challenges and Solutions

API availability and account-specific quotas

Live checks produced temporary service errors, and a Gemini 2.5 endpoint rejected generation despite appearing in the model listing. Gemini 3.6 completed a clean run but returned a five-request-per-minute free-tier limit. The pilot uses the previously configured Gemini 3.1 Flash-Lite in its own directory. Request pacing, bounded retries, provider retry hints, and resumable records reduce lost work. Availability and quota remain constraints.

False positives and incomplete executions

Earlier scoring could interpret an attacker address quoted in a body as actual delivery, and error records could enter aggregate results as unsuccessful attacks. The scorer now checks real mock destinations, excludes failed dispatches, and reports episode errors separately. Tests cover these regressions. Heuristic content and task checks remain unresolved until independent review.

Evidence continuity and honest validation

Progress Report 2 described batching and partial human review, but the supplied code did not contain a resumable runner or the human-label workflow. These features are now implemented. The midterm distinguishes that completed implementation from the still-pending labeled sample, rather than treating earlier narrative statements as validated experimental evidence.

6. Current Project Status

The project has a complete midterm implementation: a fixed agent environment, 24-attack suite, six experimental conditions, resumable and repeatable execution, automated scorers, result generation, and an independent human-validation workflow. Forty tests pass, the code is on GitHub, and all six observed clean controls complete the assigned task.

The live attack evaluation remains preliminary. At the cutoff, 4 attacked episodes have finished in the balanced pilot; the full 24-attack comparison has not been completed. The main remaining work is collecting valid observations, reviewing them independently, and turning them into supported conclusions. No human agreement, precision, or recall result is reported before human labels exist.

Midterm completion target

The target is **70% of the whole project at midterm** and **100% by Week 11**. Under the project's planning weights, research design, implementation, and software verification account for 55%; the completed pilot and preliminary analysis add 10%; the cumulative report adds 5%. These are planning estimates, not an instructor grading formula. Because the pilot is still running at the cutoff, this report does not certify that the full 70% milestone has been reached.

Interpretation is limited by a small hand-authored suite, one synthetic article, one tool-use task, one pilot trial, and a shared model family for agent, guard, and judge. Repeated trials will measure variability but will not create new independent attack designs. Credential exposure is measured inside mock tool arguments, not as real network exfiltration.

7. Plan for the Second Half of the Semester

Remaining work is scheduled to finish by **Week 11**. Core evaluation and validation take priority over optional expansion to additional attacks or models.

Target period	Tasks and milestone	Evidence to produce
Week 8	Complete the first full 144-cell attack grid and six clean controls; inspect errors and export a stratified human-review sample.	Saved transcripts, coverage and error counts, initial category-by-defense table
Week 9	Complete three trials in total: 432 attacked episodes and 18 clean controls. Finish 30-50 independent human labels.	Repeat results; labeled sample; agreement, precision, recall and ambiguity counts
Week 10	Analyze security, task completion and observed latency. Review scorer disagreements and document limitations.	Verified tables and figures; failure-case analysis; interpretation of trade-offs
Week 11	Finalize and verify the research report, presentation, demonstration, code and reproducibility instructions.	Final repository, traceable evidence, report and presentation

If quota delays persist, continue from saved checkpoints and disclose actual coverage. Optional additions, such as six more attacks or a second model, follow the core study and must not displace the promised final deliverables. Independent human labels must be supplied by a reviewer, not generated and represented as human judgments.

Evidence sources

Assignment 01 and Assignment 02 project proposal; Progress Reports 1 and 2; GitHub branch `codex/midterm-evaluation`, commit `1bac1e4`; `evidence/pytest.txt`; `docs/EXPERIMENT_PROTOCOL.md`; `docs/PROJECT_MILESTONES.md`; `results/midterm_pilot/manifest.json` and episode records; frozen report snapshot `evidence/midterm_report_snapshot.json`.

Report preparation date: October 5, 2026. Live execution can continue after the frozen cutoff; later observations are not silently included in these tables.